In [10]:
```
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score
from tensorflow.keras import layers, models
```

In [11]:
```
# Load dataset
df = pd.read_csv("ecg.csv", header=None)

# Split features and labels
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

In [12]:
```
# Split data and train only on normal ECG
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
x_train = x_train[y_train == 1]

# Normalize data
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

In [13]:
```
# Build autoencoder
encoder = models.Sequential([
    layers.Input(shape=(x_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu")
])

decoder = models.Sequential([
```

```
    layers.Input(shape=(8,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(x_train.shape[1], activation="sigmoid")
])

model = models.Sequential([encoder, decoder])
```

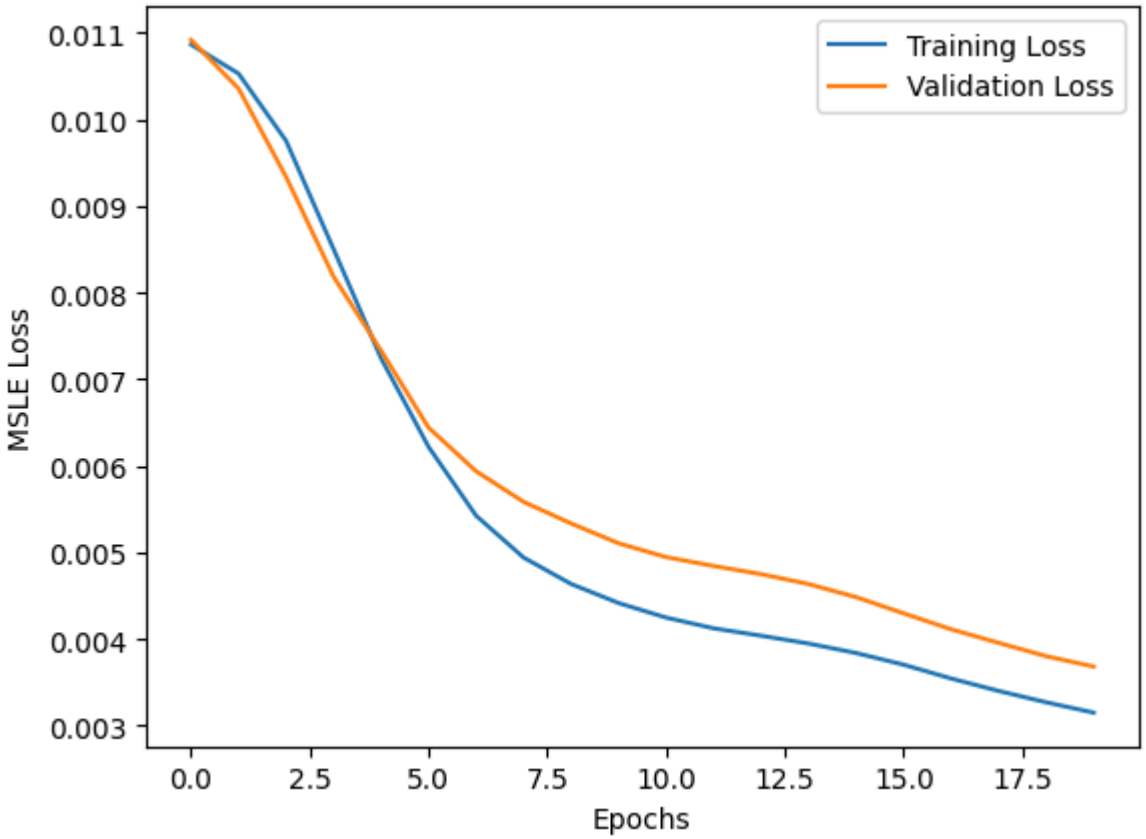
In [14]:

```
# Compile and train
model.compile(
    optimizer="adam",
    loss="mean_squared_logarithmic_error",
    metrics=["mse"]
)

history = model.fit(
    x_train, x_train,
    validation_split=0.1,
    epochs=20,
    batch_size=512
)
```

```
Epoch 1/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 60ms/step - loss: 0.0109 - mse: 0.0245 - val_loss: 0.0109 - val_mse: 0.0245
Epoch 2/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0105 - mse: 0.0238 - val_loss: 0.0104 - val_mse: 0.0233
Epoch 3/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0098 - mse: 0.0220 - val_loss: 0.0093 - val_mse: 0.0209
Epoch 4/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0085 - mse: 0.0192 - val_loss: 0.0082 - val_mse: 0.0183
Epoch 5/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0072 - mse: 0.0163 - val_loss: 0.0073 - val_mse: 0.0164
Epoch 6/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0062 - mse: 0.0140 - val_loss: 0.0064 - val_mse: 0.0144
Epoch 7/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0054 - mse: 0.0122 - val_loss: 0.0059 - val_mse: 0.0132
Epoch 8/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0049 - mse: 0.0111 - val_loss: 0.0056 - val_mse: 0.0124
Epoch 9/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0046 - mse: 0.0104 - val_loss: 0.0053 - val_mse: 0.0118
Epoch 10/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0044 - mse: 0.0099 - val_loss: 0.0051 - val_mse: 0.0113
Epoch 11/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0042 - mse: 0.0095 - val_loss: 0.0049 - val_mse: 0.0110
Epoch 12/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0041 - mse: 0.0092 - val_loss: 0.0048 - val_mse: 0.0107
Epoch 13/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0040 - mse: 0.0090 - val_loss: 0.0047 - val_mse: 0.0105
Epoch 14/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0039 - mse: 0.0088 - val_loss: 0.0046 - val_mse: 0.0103
Epoch 15/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0038 - mse: 0.0086 - val_loss: 0.0045 - val_mse: 0.0099
Epoch 16/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0037 - mse: 0.0083 - val_loss: 0.0043 - val_mse: 0.0095
Epoch 17/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0035 - mse: 0.0080 - val_loss: 0.0041 - val_mse: 0.0091
Epoch 18/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0034 - mse: 0.0076 - val_loss: 0.0040 - val_mse: 0.0088
Epoch 19/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0033 - mse: 0.0073 - val_loss: 0.0038 - val_mse: 0.0085
Epoch 20/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0031 - mse: 0.0071 - val_loss: 0.0037 - val_mse: 0.0082
```

In [15]:
```
# Plot training and validation loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSLE Loss")
plt.legend()
plt.show()
```



In [16]:
```
# Calculate threshold using normal training data
predictions = model.predict(x_train, verbose=0)
errors = tf.keras.losses.msle(x_train, predictions).numpy()
threshold = np.mean(errors) + np.std(errors)
```

In [17]:
```
# Detect anomalies in test data
predictions = model.predict(x_test, verbose=0)
errors = tf.keras.losses.msle(x_test, predictions).numpy()

anomalies = (errors > threshold).astype(int)
y_pred = 1 - anomalies

print("Threshold:", threshold)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Number of anomalies:", np.sum(anomalies))
```

```
Threshold: 0.007528793
Accuracy: 0.96
Number of anomalies: 454
```

In [ ]:
```

```